Nakha (■■■■■)

Architecture & Deployment Audit

Prepared: May 2026 · Platform: Home-cooked food ordering, Bechar, Algeria

Executive Summary: Nakha is a fully serverless React SPA backed by Firebase Firestore + Authentication and hosted on Vercel. The architecture choice is correct. The three critical gaps are: (1) client-side price/commission calculations with no server validation, (2) credentials hardcoded in source, and (3) three financial operations that are non-atomic. Fix those and the app is production-ready at near-zero cost.

1. Tech Stack

Layer	Technology	Version	Notes
Frontend framework	React + Vite	React 19.2 / Vite 8.0	SPA, no SSR, RTL
Backend framework	None	—	Fully serverless
Database	Firebase Firestore	SDK v12, eur3 region	NoSQL, 13 collections
Auth	Firebase Authentication	SDK v12	Email/password only
Image storage	Cloudinary	REST API	Unsigned preset, browser-direct
Hosting	Vercel	vercel.json present	SPA rewrite configured
Monitoring	Vercel Speed Insights	v2.0 installed	NOT wired up in code yet
Icons	Lucide React	v1.8	
Animations	Framer Motion	—	
State persistence	localStorage	—	cart, favorites, sound prefs

No backend server. No Cloud Functions. No queue. No cron. No worker process.

2. Deployment Architecture

Is Vercel-only suitable?

YES — for now. Every piece of logic runs either in the browser or inside Firebase/Cloudinary. There is no process that needs persistent compute.

Can frontend and backend be separated?

There is no backend to separate. The backend is Firestore Security Rules combined with client-side SDK calls. The gap is that 3 critical operations (order creation, commission deduction, topup approval) currently run client-side and should become Cloud Functions — still serverless.

Part	Serverless-friendly?	Where it runs
Auth flows	YES	Firebase Auth SDK (browser)
All DB reads/writes	YES	Firestore SDK (browser)
Image uploads	YES	Cloudinary REST from browser
Commission deduction	YES — but UNSAFE	runTransaction in browser (should be Cloud Function)
Cart	YES	localStorage only
Order total validation	MISSING	Does not exist — client sends price unchecked

3. Firebase Usage

Collections (13 total)

```
users · cooks · customers · dishes · orders · ratings · transactions · topup_requests ·
invite_codes · invite_tokens · invite_analytics · beta_whitelist · waitlist
```

Real-time listeners — 5 active per cook session

File	What is watched	Risk
CookDashboard.jsx	Cook profile doc (balance, status)	Reads on every profile change
CookDashboard.jsx	Dishes for cook	Reads on every dish change
CookDashboard.jsx	Orders for cook	Reads on every order change
CookOrders.jsx	Orders for cook (duplicate)	Duplicate listener — consolidate
useOrderNotifications.js	Pending orders for audio chime	Always-on for cooks

Billing concern: Each onSnapshot keeps a persistent WebSocket connection. 5 listeners per active cook session means every active cook burns 5× the expected reads. At high traffic this is the #1 cost driver.

Transactions — atomic operations (done right)

File	Operation
CookOrders.jsx	runTransaction for commission deduction
RateOrder.jsx	runTransaction for rating aggregation

Non-atomic writes (risk)

File	Problem	Severity
ManageTopups.jsx:74-99	Topup approval = 3 separate updateDoc/addDoc calls — crash mid-sequence leaves inconsistent balance	HIGH
ManageCooks.jsx	Cook deletion = 2 separate deleteDoc calls — can orphan the users doc	HIGH

Scaling / Billing Risks

Spark free plan limits: 50k reads/day, 20k writes/day, 1 GB storage.
- **No pagination anywhere** — Home.jsx reads ALL cooks + ALL dishes in one shot.
- **Medium traffic** (50 cooks, 200 dishes, 100 orders/day): stays in free tier.
- **High traffic** (200+ cooks, 1000+ dishes): needs Blaze pay-as-you-go.

4. Backend Workload

Workload Type	Present?	Notes
Cron jobs	NO	—
Background queues	NO	—
Scheduled tasks	NO	—
Webhooks	NO	—
Web scraping	NO	—
Long-running processes	NO	—
File processing	NO	Cloudinary handles all image transforms
AI tasks	NO	—
Email / SMS	NO	—
Payment gateway	NO	Manual BaridiMob transfer + admin approval

Conclusion: Zero backend workload. No VPS, Railway, Render, or persistent server of any kind is needed.

5. Hosting Recommendation

Final answer: Vercel + Firebase Blaze + 3 Cloud Functions

- **Vercel:** hosts the static SPA — already configured, zero cost at current scale.
- **Firebase Blaze:** Spark (free) plan blocks Cloud Functions entirely. Blaze is pay-as-you-go — at current traffic it costs $0 extra.
- **3 Cloud Functions:** createOrder, markOrderReady, approveTopup — these own all financial logic server-side.

Do NOT add:

Option	Why not
VPS / Railway / Render	Nothing runs persistently — wasted cost
Separate Node.js server	Adds ops complexity for zero benefit
Google Cloud Run	Overkill — Cloud Functions is sufficient
AWS	No reason to leave Firebase ecosystem

6. Production Readiness Audit

CRITICAL — blockers (fix before any real user)

Issue	File	Fix
Firebase config hardcoded in source	src/firebase/config.js	Move to VITE_FIREBASE_* env vars in Vercel dashboard
No .env in .gitignore / no .env.example	.gitignore	Add .env, .env.local entries; create .env.example
Client sends totalPrice — no server validation	Checkout.jsx:288	Cloud Function recalculates price from dish IDs + qty
Commission deduction uses client-supplied price	CookOrders.jsx:113	Cloud Function must own this calculation
Orders publicly readable — customerPhone exposed	firestore.rules:148	Restrict reads to order owner or authenticated users
Admin phone + bank RIP hardcoded in source	src/config/settings.js	Move to environment variables

HIGH — fix before public launch

Issue	File	Fix
No rate limiting on any operation	—	Firebase App Check + Cloud Function guards
Topup approval: 3 non-atomic writes	ManageTopups.jsx:74-99	Rewrite as writeBatch or Cloud Function
Cook deletion leaves orphan users doc	ManageCooks.jsx	Use writeBatch for both deleteDoc calls
No error monitoring	—	Add Sentry (npm i @sentry/react, 3 lines in main.jsx)
Vercel Speed Insights not used	App.jsx	Add component
No CSP headers	vercel.json	Add headers block with Content-Security-Policy
No HTTPS redirect	vercel.json	Add HTTP→HTTPS redirect rule
Early access bypass is client-side only	App.jsx:73	sessionStorage.nakha_bypass = '1' bypasses gate via DevTools

MEDIUM — fix within 30 days post-launch

Issue	File	Fix
No pagination — Home reads ALL cooks + dishes	Home.jsx	Add limit() + infinite scroll
Rating system allows unauthenticated spam	firestore.rules	Require order ownership proof in rules
Invite codes/tokens publicly readable	firestore.rules:245-282	Restrict to authenticated users
Cart prices not re-fetched from Firestore at checkout	CartContext.jsx / Checkout.jsx	Fetch live prices in Cloud Function before order write
Phone masking inconsistent	MyOrders.jsx:229	Audit all phone display locations
No input sanitization	All forms	Add DOMPurify for comment/notes fields

Missing entirely

Missing piece	Impact
Firestore automated backups	Data loss with no recovery path
Firebase App Check	Bots can abuse Firestore quota freely
Admin audit log collection	No record of who approved which financial operation
Additional composite indexes	Queries fail silently at scale without explicit indexes
Production logging strategy	console.error() is invisible in production

What is done right

✓ firestore.rules exists with real RBAC — cooks can't set their own balance or escalate to admin
✓ runTransaction used correctly for commission deduction and rating aggregation
✓ firestore.indexes.json has a composite index for order history queries
✓ vercel.json SPA rewrite is correctly configured
✓ Commission rate (9%) and phone masking logic are present in business rules
✓ Max negative balance guard prevents infinite overdraft

7. Estimated Monthly Cost

Low traffic — 10 active cooks, 50 orders/day

Service	Cost
Vercel (Hobby)	$0
Firebase (Spark free tier)	$0
Cloudinary (free tier)	$0

Service	Cost
TOTAL	~$0 / month

Medium traffic — 50 cooks, 300 orders/day

Service	Cost
Vercel (Hobby)	$0
Firebase Blaze (~500k reads/day)	~$3–8 / month
Cloudinary (50–100 uploads/day)	~$0–10 / month
Sentry (free tier)	$0
TOTAL	~$5–20 / month

High traffic — 200+ cooks, 2000 orders/day

Service	Cost
Vercel (Pro)	$20 / month
Firebase Blaze (~5M reads/day)	~$50–150 / month
Cloud Functions invocations	~$5–15 / month
Cloudinary (Plus plan)	$89 / month
Sentry (Team)	$26 / month
TOTAL	~$200–300 / month

Biggest cost driver at scale: The 5 onSnapshot listeners per cook session. Each change to a watched collection triggers a read for every connected cook. Consolidating to 1–2 listeners cuts Firestore read costs ~60%.

8. Final Recommendation

Use: Vercel + Firebase Blaze + 3 Cloud Functions + Sentry

Action Plan (in priority order)

1 — Immediately

Move Firebase config + admin phone + bank RIP to Vercel environment variables (VITE_FIREBASE_*). Add .env to .gitignore. Create .env.example. Upgrade Firebase project to Blaze — costs nothing at low volume.

2 — Before first real transaction

Write 3 Cloud Functions: • createOrder — fetch dish prices server-side, validate totals, write order. • markOrderReady — own commission deduction server-side (most critical). • approveTopup — atomic batch: update cook balance + mark request + log transaction.

3 — Before public launch

Add Sentry (3 lines in main.jsx). Activate Vercel Speed Insights. Add CSP + HTTPS headers to vercel.json. Restrict order reads in firestore.rules to authenticated users.

4 — First month after launch

Add pagination to Home.jsx. Enable Firebase App Check. Set up Firestore scheduled export (daily backup). Consolidate onSnapshot listeners in CookDashboard.

The architecture is correct and the Vercel + Firebase choice is right. The gaps are not design problems — they are 3 missing Cloud Functions and missing environment variable hygiene. Fix those and Nakha is production-ready at a running cost of effectively $0 for the first 6–12 months of operation.